

Final Project

For the final project, you are to develop a 32-bit CPU design using the provided instruction set.

To test the performance of your CPU design, you are to use a program based on the “Sieve of Eratosthenes” – a method for finding prime numbers by removing all non-primes¹. You have been provided with an assembly code performs this algorithm on numbers in the range of 2 and 127. The result of this calculation is to be transmitted from a memory mapped IO port.

Aim:

Develop a 32-bit single or multi-cycle CPU capable of performing a search for prime numbers.

CPU

This topic has covered both the design and implementation of computer architecture. In the final project all these parts will be brought together to produce a full CPU design. This CPU will be a 32-bit design based on a subset of the MIPS instruction set using your choice of either a single-cycle or multi-cycle implementation. The basic structure of the CPU should follow the design shown in chapter three of your Computer Organisation notes. A multi-cycle machine should include a finite state machine corresponding to the five states shown on page 4 of chapter 3.

The entity description of your design should have the following ports;

- clk – a clock line
- ioaddress – the address that the data should be sent to
- iodata – the data to be transmitted
- ioenable – a write enable line

The last three ports are used to address the memory mapped IO described in the memory section.

Instruction Set

The instruction set is based on the MIPS instruction set. You are required to implement R, I and J format instructions as specified in appendices 1 and 2. Note that all memory accesses including instruction fetch are 32-bits wide and performed on 32-bit boundaries. The program counter will treat each 32-bit instruction as a single location.

Memory

Unlike MIPS, this design will use the Harvard architecture with separate program and data memories. You should create memory units matching the following specifications:

- Instruction memory unit – A single read port memory 1024 32-bit locations in size. All access will be on 32-bit boundaries (ROM)
- Single read port – single write port memory (RAM) that can store up to 512 bytes accessed on 32-bit boundaries.
- Memory mapped IO – 512 bytes on 32-bit boundaries. This represents a series of registers that are implemented externally to the CPU but share the same address bus, read, and write ports as the RAM module. A write enable signal and a multiplexor should be configured such that addresses 0 to 127 refer to RAM, while 128 to 255 refer to the external IO. Your

¹ https://www.encyclopediaofmath.org/index.php/Eratosthenes,_sieve_of

CPU component should include ports connecting these signals to a toplevel component, but the IO registers themselves do not need to be implemented. A block diagram of this memory can be seen in Figure 1.

- Dual read port – single write port, 32-bit wide register bank containing 16 registers.

ALU

The ALU will be a 32-bit design based on the one developed in Practical 2.

Architecture of Memory and IO

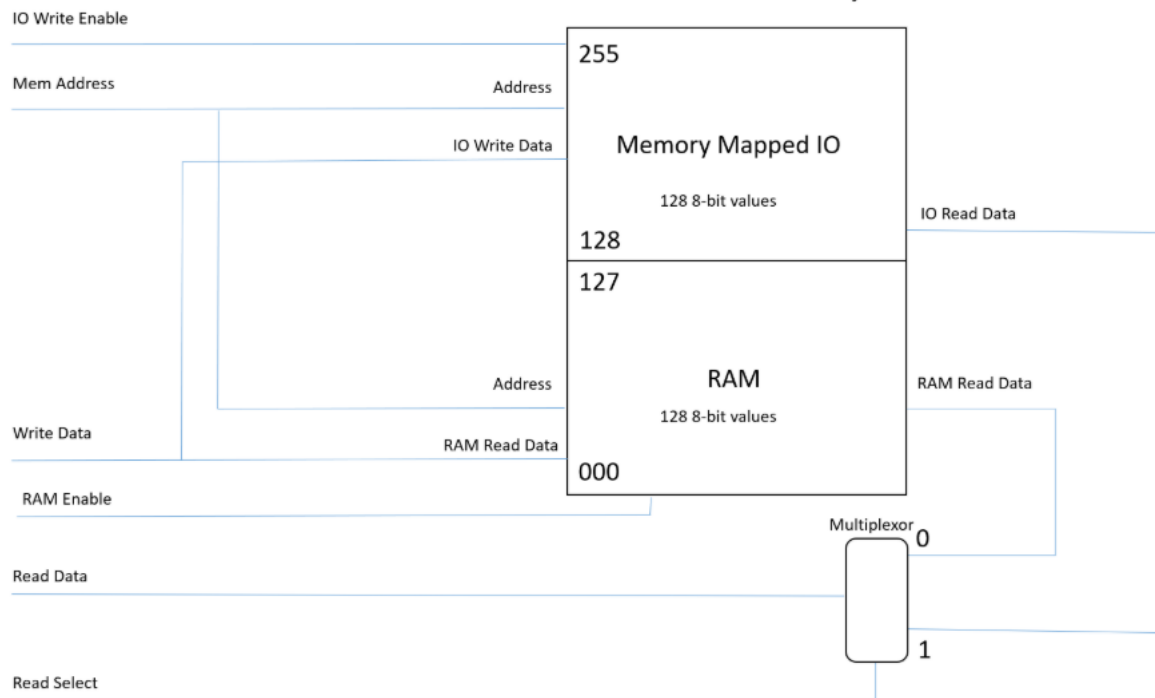


Figure 1 – Block Diagram of the Memory Subsystem. Memory access in the range of 128-255 should be directed to the external memory mapped IO.

GE Students (ENGR 9781)

In addition to the CPU specified here, GE students are to implement addition in their ALU using a carry-lookahead adder.

Block Diagram

Students should begin the project by drafting a block diagram showing the relationship between the modules of their chosen CPU design. This can be drafted electronically or hand drawn and then scanned into the logbook.

Testing

You have been provided with an assembly language implementation of the Sieve of Erasthones program in Appendix 3. You are to convert this to machine code and use the result to test the correct operation of your CPU.

To demonstrate the expected operation, a python implementation of the algorithm can be seen in Appendix 4, with the result seen in Appendix 5. This implementation can be used to illustrate the operation of the algorithm, and also to use as a cross-check with the operation of your design.

Report

The report should be laid out in the manner of a formal report. You may use the Scientific Lab Report outline from the student learning centre for a guideline:

http://www.flinders.edu.au/slc_files/Documents/Blue%20Guides/Sc%20Lab%20Reports.pdf

This page from Penn State University would also be useful:

<http://www.writing.engr.psu.edu/workbooks/reports.html>

Note that the front matter of your report should include the following:

- Title page
- Abstract
- Acknowledgements
- Table of contents
- Table of illustrations

The method section should include a description of the VHDL code and software that you have developed. This section should also note the origin of any code that you have included from external sources.

Your discussion section should include a high level overview and analysis of what you have achieved in this project. You may also use this section to consider possible alternative development strategies and applications for your design. Potential strategies for improving performance for the provided task would also be relevant.

You may include diagrams and images from external sources, but they must be cited appropriately.

The appendices should include any pseudocode, software listings and testbench output including the state of registers and memory at the completion of your program.

You should include a bibliography using either APA or IEEE style citations.

For further assistance with language and formatting, the library provides a drop in centre for writing assistance: <http://www.flinders.edu.au/current-students/slc/services/learning-lounge.cfm>

Deliverables

The deliverables for this project comprise three items:

- Source code for your design
- Your project log
- A formal report

Submission of project reports will use FLO's electronic submission mechanism. Submit PDFs of your report and log, along with a .zip file containing your VHDL project to the FLO assignment titled "Project". This will be accessible on or before Monday of week 12. The due date is 5pm on Monday of Week13. Later submissions will be allowed, **but will incur penalties as per the SAM.**

Appendix 1 - Instruction set

Name	Instruction type	Opcode (6 bits)	Funct (3 bits)	description
add	R	0x00	000	$\$rs + \$rt \rightarrow \$rd$
sub	R	0x00	001	$\$rs - \$rt \rightarrow \$rd$
and	R	0x00	010	$\$rs \text{ and } \$rt \rightarrow \$rd$
or	R	0x00	011	$\$rs \text{ or } \$rt \rightarrow \$rd$
xor	R	0x00	100	$\$rs \text{ xor } \$rt \rightarrow \$rd$
not	R	0x00	101	not $\$rs \rightarrow \rd
lbs	R	0x00	110	lbs $\$rs \rightarrow \rd
inc	R	0x00	111	$\$rs + 1 \rightarrow \rd
load immediate	I	0x22	XXX	immediate $\rightarrow \$rt$ (15 downto 0)
load	I	0x23	XXX	mem($\$rs + \text{immediate}$) $\rightarrow \$rt$
store	I	0x21	XXX	$\$rt \rightarrow \text{mem}(\$rs + \text{immediate})$
beq	I	0x05	XXX	If $rs == rt$ then $\$pc = \$pc + \text{immediate}$ (9 downto 0)
bne	I	0x04	XXX	If $rs \neq rt$ then $\$pc = \$pc + \text{immediate}$ (9 downto 0)
jump	J	0x02	XXX	$\$pc = \text{jump target}$
nop	J	0x3f	XXX	No operation

X = Don't Care

Appendix 2 – Instruction format

Instruction type	31					
------------------	----	--	--	--	--	--

Appendix 3 – Sieve of Erasthones algorithm

```
--r0 - zero
--r1 - one
--r2 - working value
--r3 - target
--r4 - prime offset
--r5 - loop counter
--r6 - temp1
--r8 - sign test
load immediate r0 0
load immediate r1 1
load immediate r2 1
load immediate r3 127
load immediate r4 128
load immediate r8 2048

--Set the contents of working memory to 1
#label1
store r2 r1 0
inc r2 r2 r2
bne r2 r3 #label1

--Start the algorithm at index 1. Finish at 513

load immediate r2 1
#label2
inc r2 r2 r2
beq r2 r3 #label4
load r6 r2 0
bne r6 r1 #label2

--Send prime to the IO bus
store r4 r2 0
inc r4 r4 r4

--Clear all multiples of the prime
load immediate r6 0
add r6 r2 r6
#label3
add r6 r2 r6

--The calculated index may exceed the maximum value, but we lack a
branch on greater than. Subtract and check to see if the number is
negative
--this could be performed by checking the msb, but here we only test
bit 11 - which should be more significant than our mathematical
operations.
sub r6 r3 r7
and r7 r8 r7
beq r7 r8 #label2

--clear the non-prime value
store r6 r0 0
jump #label3
```

```
--Loop endlessly  
#label4  
jump #label4
```

Appendix 4 – Python implementation

```
#!/usr/bin/python

import numpy
n= 128
numbers=numpy.ones (n, dtype=bool)

for i in range(2,n):
    if numbers[i]:
        temp=i+i
        while temp<n:
            numbers[temp]=0
            temp=temp+i

print "prime number in decimal"
print numpy.where(numbers>0)

numpy.set_printoptions(formatter={'int':hex})
print "prime number in hexadecimal"
print numpy.where(numbers>0)
```

Appendix 5 – Output of Python implementation

```
prime number in decimal
(array([ 0,  1,  2,  3,  5,  7, 11, 13, 17, 19, 23, 29, 31,
        37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89,
        97, 101, 103, 107, 109, 113, 127]),)
prime number in hexadecimal
(array([0x0L, 0x1L, 0x2L, 0x3L, 0x5L, 0x7L, 0xbL, 0xdL, 0x11L, 0x13L,
        0x17L,
        0x1dL, 0x1fL, 0x25L, 0x29L, 0x2bL, 0x2fL, 0x35L, 0x3bL, 0x3dL,
        0x43L, 0x47L, 0x49L, 0x4fL, 0x53L, 0x59L, 0x61L, 0x65L, 0x67L,
        0x6bL, 0x6dL, 0x71L, 0x7fL]),)
```